

Gestion de projet:

Pour gérer ce projet, j'ai utilisé Notion comme outil de suivi des tâches. J'ai organisé les fonctionnalités en colonnes Kanban : Backlog, En cours et Terminé. Chaque fonctionnalité du sujet ECF correspondait à une carte dans le board.

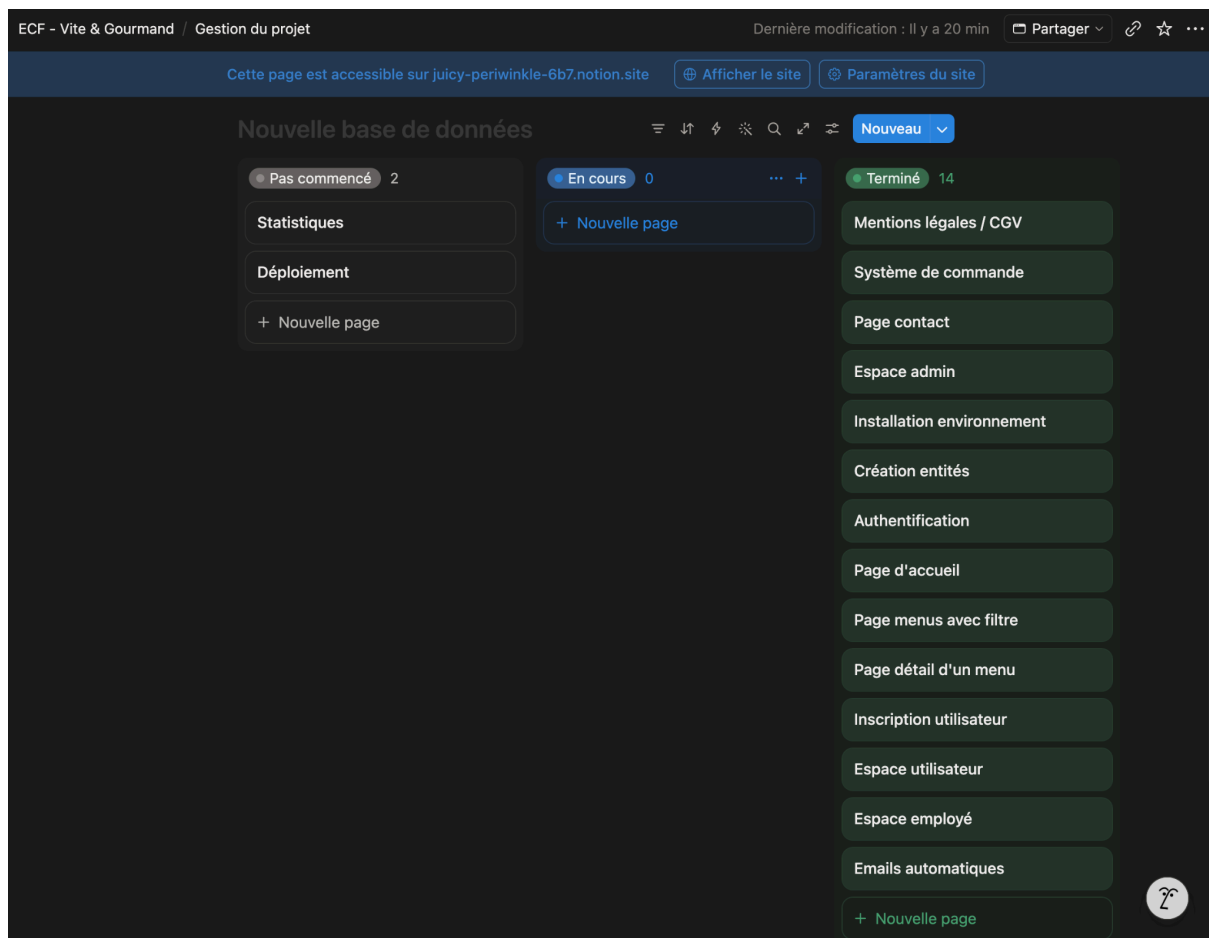
Côté développement, j'ai appliqué les bonnes pratiques Git :

Une branche "main" pour la version stable.

Une branche "develop" pour l'intégration.

Une branche "feature/xxx" par fonctionnalité.

Cette organisation m'a permis de travailler de manière structurée et de garder un historique clair de l'avancement du projet.



Réflexions initiales technologiques:

Symfony

J'ai choisi Symfony comme framework PHP car c'est un framework professionnel et robuste qui propose une architecture MVC bien structurée. Il offre des composants intégrés très utiles pour ce projet : Doctrine pour la base de données, Security Bundle pour l'authentification, Mailer pour les emails et Twig comme moteur de templates.

MySQL

J'ai choisi MySQL comme base de données relationnelle car c'est la solution la plus utilisée dans l'écosystème PHP/Symfony. Elle est gratuite, open source et très bien supportée par Doctrine ORM.

Bootstrap

J'ai choisi un template Bootstrap comme framework CSS car c'est le framework le plus utilisé en entreprise. Il offre un système de grille responsive et une grande bibliothèque de composants prêts à l'emploi.

Configuration de l'environnement:

Système d'exploitation : macOS

Outils installés :

- PHP 8.5.3
- Composer 2.9.7
- Symfony CLI 5.17.1
- MySQL 9.6.0
- Git
- Visual Studio Code

Extensions VS Code :

- PHP Intelephense

- Twig Language 2

Outils de gestion de base de données :

- phpMyAdmin

Outils de gestion de projet :

- Notion
- GitHub

J'ai utilisé le serveur intégré de Symfony CLI et Mailtrap comme serveur SMTP de test.

Modèle Conceptuel de Données:

Le MCD représente la structure de la base de données avec toutes les entités et leurs relations.

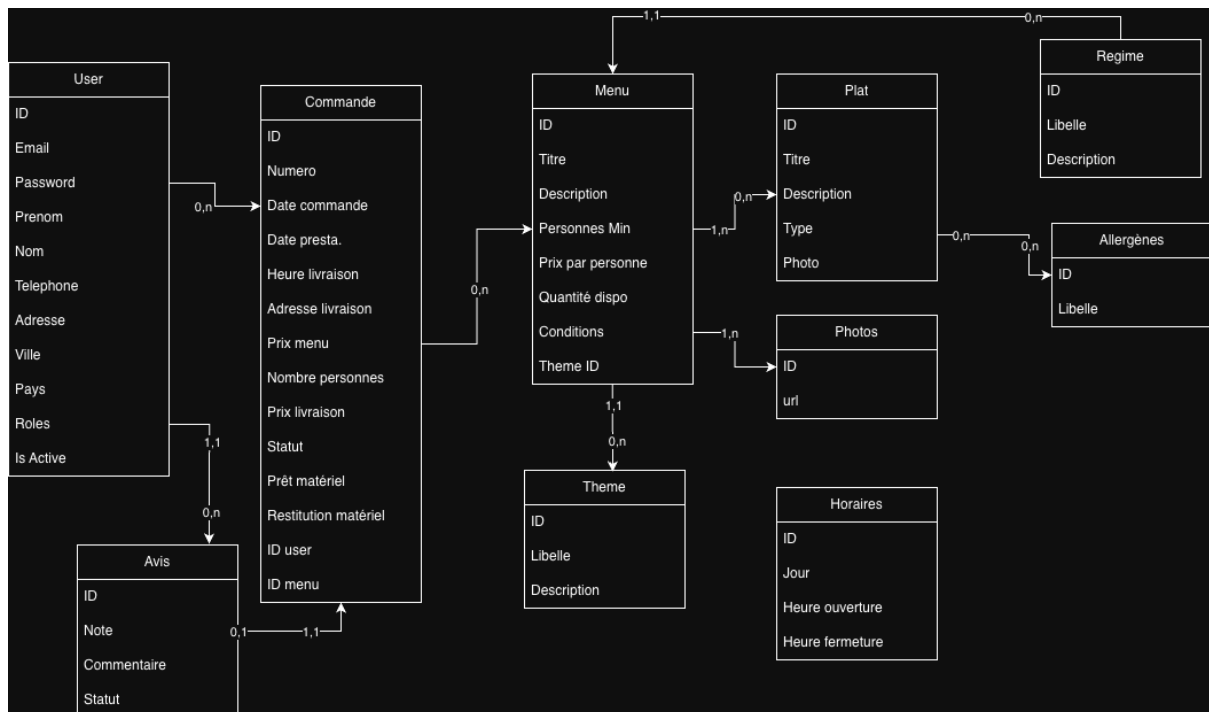


Diagramme d'utilisation:

Le diagramme d'utilisation montre les actions possibles pour chaque rôle.

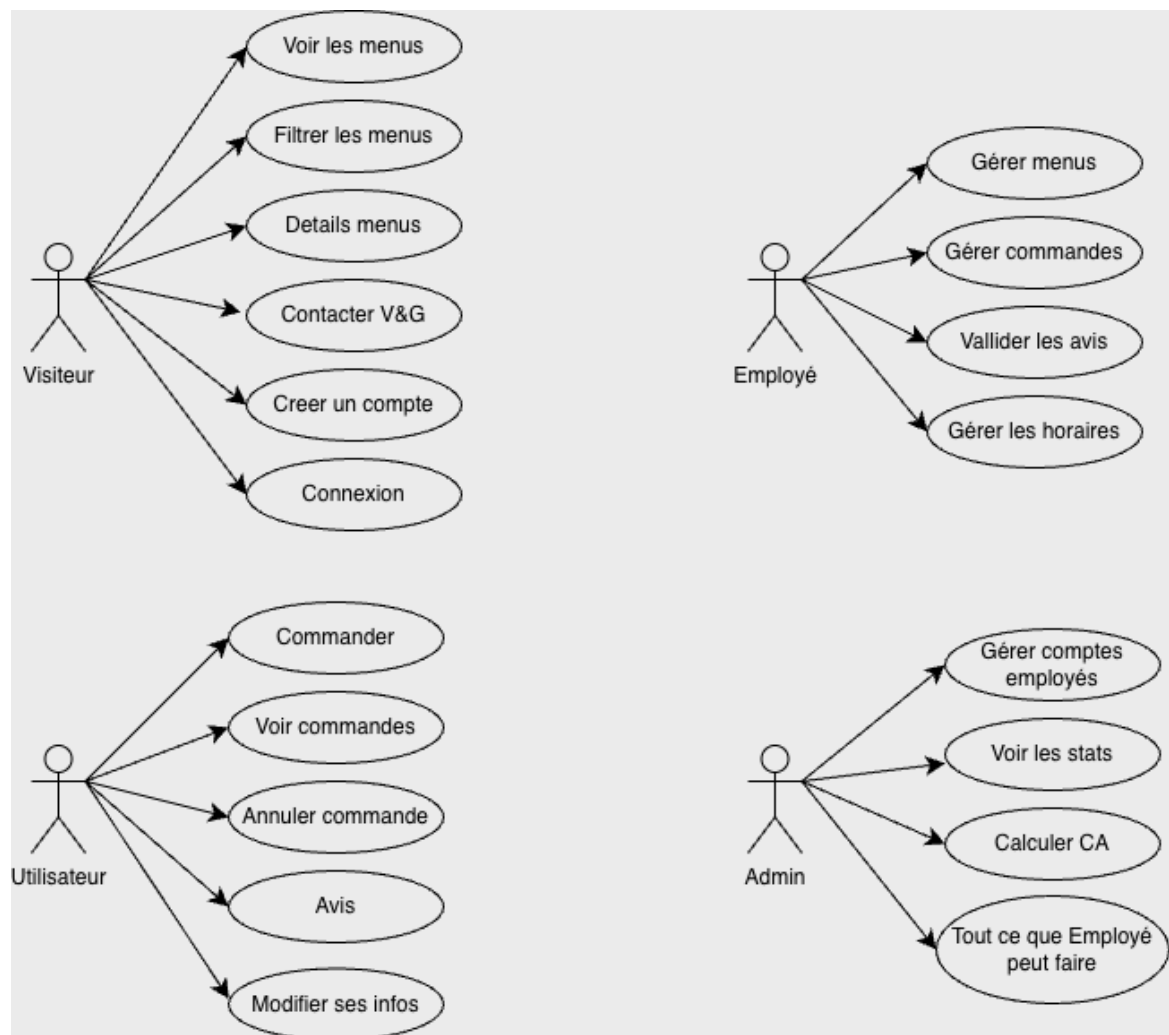
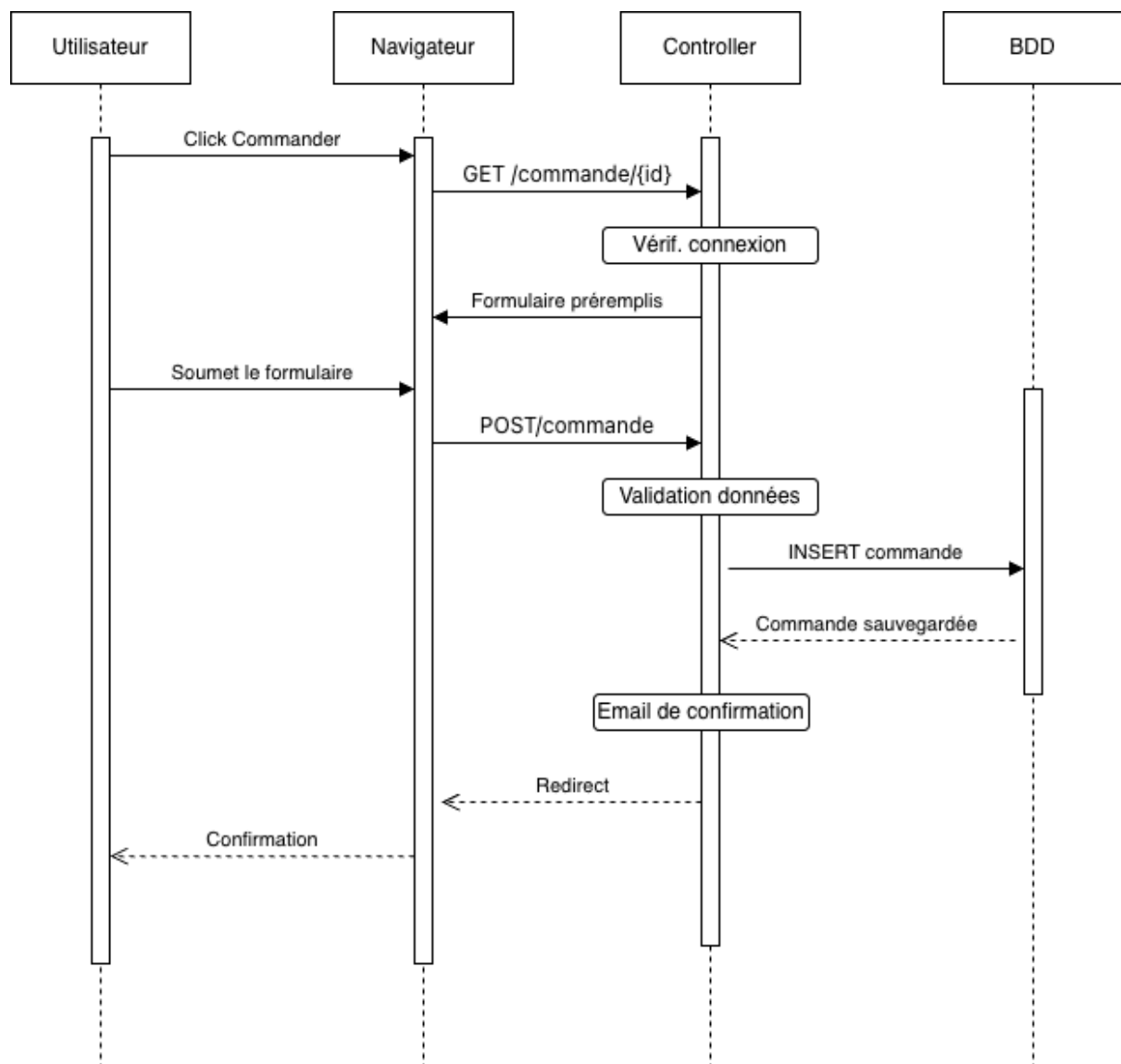


Diagramme de séquence:

Le diagramme de séquence montre le flux d'une commande d'un menu, de la sélection jusqu'à la confirmation par email.



Documentation du déploiement:

J'ai choisi fly.io comme plateforme de déploiement.

J'ai installé flyctl qui est un outil de commande fly.io.

Fly.io nécessite un Dockerfile pour construire l'image de l'application. Ce fichier décrit l'environnement d'exécution :

```
FROM php:8.4-cli

RUN apt-get update && apt-get install -y \

    git zip unzip curl libssl-dev pkg-config libpq-dev \

    && pecl install mongodb \

    && docker-php-ext-enable mongodb \

    && docker-php-ext-install pdo pdo_pgsql

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

ENV COMPOSER_ALLOW_SUPERUSER=1

WORKDIR /app

COPY . .

RUN composer install --no-dev --optimize-autoloader --no-scripts \
    --ignore-platform-req=ext-mongodb

ENV APP_ENV=prod

EXPOSE 8080

CMD php -S 0.0.0.0:8080 -t public/
```

Le déploiement se fait très simplement avec les commandes:

```
flyctl launch
```

```
flyctl deploy
```

L'application est accessible à l'adresse :

<https://vite-et-gourmand-moonlit-fog-6945.fly.dev>